Winter Milestone Compliance Report 4

GK02: Heterogeneous CPU-GPU Implementation of Collision Detection for Gaming

Project Manager:
JJ Marr - 500879103

Team Members:
Victor Do - 501137174
Cameron Tuffner-Lyons - 501029609
Davis Cheung - 501014636

Faculty Supervisor
Dr. Gul N. Khan

April 3rd, 2025

Abstract:

This milestone compliance report details the progress made on the fluid simulation capstone project. Significant advancements have been achieved in core simulation features, including the integration of a unified physics shader for optimized performance, the addition of new gravity modes and particle friction properties to simulate powders and solids, and the development of new fluid types. Game content has been expanded with the creation of multiple levels, each designed to showcase specific simulation capabilities, such as temperature mixing, fluid state changes, and object manipulation. Additionally, new game scripts and script enhancements were implemented to add extra functionality, such as snapping objects, cloning objects using factory scripts, and fluid type detection. Challenges encountered include script refactoring to maintain code consistency and address unforeseen conflicts between new and existing functionality. Web application performance issues were encountered on iOS devices.

Individual Member Contributions (For the whole project):

Student Name:	Github and Functional Tasks Completed
Student A: (VD)	- Made Simulation Scripts Hot Swappable for faster development - Added Input Manager for player interactivity - Added Level Manager for level progression, and score tracking - Worked on the UI Elements, for main menu, level selection screen, pause menu, sandbox level - Created the side & bottom UI carousels for opening submenus and selecting fluid brushes. And added tooltips to explain UI elements on hover. Also added the mouse circle and strength indicators reused in each level. - Created Github Readme document and Kanban board for Git Task Tracking - Created benchmarking scripts and added FPS display counter to the top left - Created Scriptable Object System for generating new fluid types - Added Settings Menu for adjustable screen resolution and audio settings - Created Initial Fluid Density Sensor for CPU readback of GPU particle buffers - Helped refactoring particle data to use Array of Structs then Benchmarked results for different script variations - Fixed instability issues with disabling particles. And added random # generator to randomize particle positions when disabled for better distribution / performance - Added / Edited Kernels to Spawn particles. - Created source and drain objects to spawn and delete particles without user input - Added initial velocity and fluid type parameters to source object - Refactored structs and enums to their own classes so that they could be shared between different sim scripts - Refactored to remove garbage creation to reduce need for garbage collection - Brought counting sort version of simulation script up to date with features before deprecating bitonic sort scripts - Implemented brush to draw fluid particles into the scene in realtime. - Added Different Brush Types (Nothing, Drawing, Gravity Manipulation, Eraser) - Created Lava, Alcohol, Beer, Cold Fire, Crude Oil, Multiple Fire Types for each flammable fluid, Death Ray, Honey, Ion Blast, Neutron, Oil, Plutonium, Smoke, Steam, and Water fluid types. - Added Friction kernels and powder types like: Cement, Concrete, Stone, Sediment, and Snow - Added Simulation Edge settings (void, loop, solid) and Gravity modes (normal, reverse, left, right, radial, zero) - Worked on making simulation behavior more consistent across different frame rates. - Worked on Visual Shader System for all fluids (velocity based color, glowing, temperature based, fuzzy edges) - Responsible for managing live website with updated builds - Refactored and Bug fixed multi-fluid particle physics interactions - Added debugging option to refresh visual shader information while running for faster development - Added dragging, resizing and rotating scripts for obstacles - Swapped from sprites to line renderer outlines for obstacles for better scaling - Made a global fluid ID system, removed fluid Index system - Planned out 10 level ideas. Completed Levels 1, and 5. Helped with polishing levels 2, 3, 4, 5, 6. Working on 7-10. - Added blur shader, and gradient shaders for UI elements. Added CPU particle systems for visual effects. - Bugfixed temperature kernels not working in conjunction with thermal boxes - Fixed issues where particle would clump together due to viscosity, or pressure forces - Freed GPU memory after buffer creation and discarding to stop memory leaks - Added feature to change brush size and strength - Added obstacle spawn system and context menus to enable changing obstacle properties in realtime - Added simulation setting adjustment menu to change settings like fixed time step, gravity, edge mode on the fly. - Unified 5 different kernel dispatches for physics into 2 kernels for better performance. - Added System to scale the sandbox simulation to different hardware levels. - Added obstacle cleanup to destroy obstacles which fall outside the simulation bounds. - Capstone Poster, MCR Reports, EDP Reports, editing for trailer, pitch videos and poster presentation video.

Student B: (JM)	• Lava level • Introduced the natural gas and gas fire fluid types • Completed level 4 (lava level) • Vary particle size based on density in shader • Fixed locally-hosted server.js script • Failed at debugging WebGPU lag on WebKit browsers (turns out WebKit doesn't support WebGPU on Linux yet)
Student C: (CTL)	- CPU-GPU Single-fluid split - Performance Benchmarking - Unified Shader Performance Benchmarks - ASync GPU memory readback implementation. - Level 2 - Carnival game - Level 4 - Puzzle Level - Level 6 - Casting Level - Oral Presentation Slides - Various new fluids - Capstone Poster - MCR Reports - EDP Reports
Student D: (DC)	- Multiple fluid type support, FluidType, FluidParam, and FluidData structs - Per-particle simulation for multiple fluid type and cross-fluid type interactions - Temperature physics kernel - Temperature-changing box colliders (ThermalBoxes) - Thermal entropy simulation - Backporting of thermal and entropy logic to CPU-GPU script variants - Simulation refactors for Struct of Arrays, Array of Structs - Simulation refactor for counting sort - Hash optimization/refactor from 3 values to 1 - Particle writeback optimizations - Backporting of SoA, AoS, counting sort, 3-to-1, and writeback optimizations to CPU-GPU script variants - Particle state change implementation + state change kernels - Support for multiple different fluid spawners of different fluid types in a stage - Thermal sensors for temperature detection - Sensor managers w/ automatic scanning for reduced particle readback - Various new fluids - Snapper script for snapping game objects together - CupFactory script for generating clones of cup objects - FluidDetector refactor for fluid type detection - Drink level development - (WIP) floating box colliders - Bug fixes (various) - Change/PR review - Oral Presentation Slides - Capstone Poster - MCR Reports - EDP Reports

Table 1: This is a breakdown of the tasks each member worked on **for the entire project.**

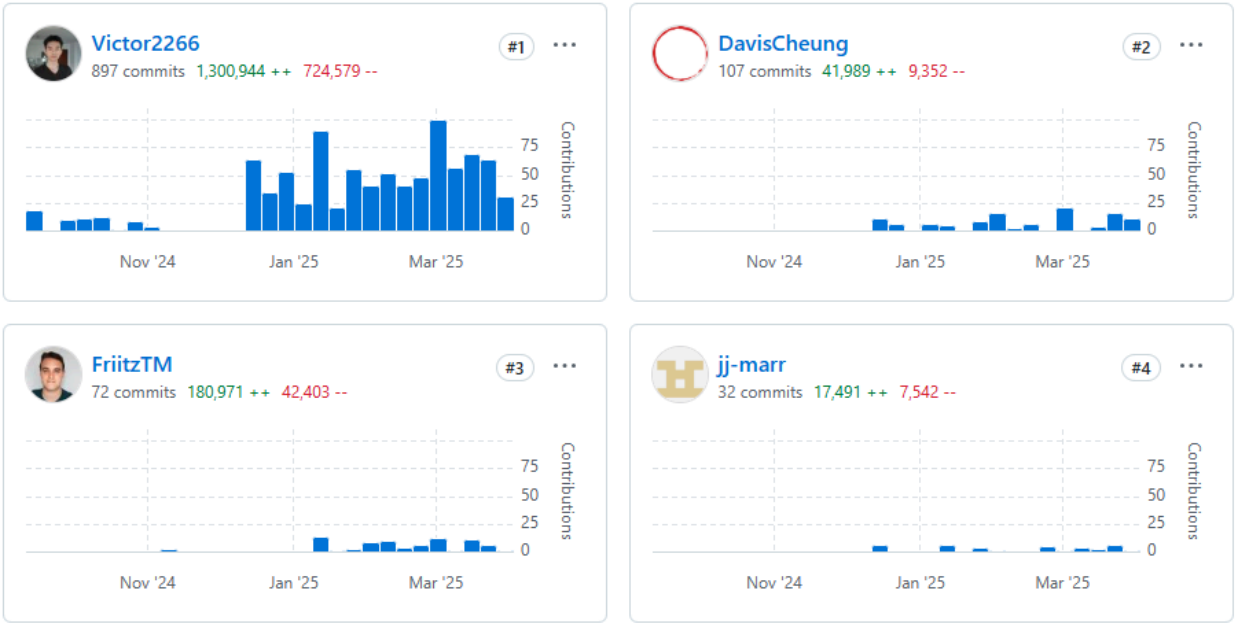


Figure X: This is a breakdown of the commit history of each member over the whole project.

Tasks Outlined & Group Member Contributions (For This MCR Period):

Student Name:	Github and Functional Tasks Completed	Report Sections
Student A: (VD)	- Added gravity modes - Modified gas fuel type - combine all kernels with nearest neighbour search into one kernel (Optimization) - Added non-fluid powders with friction kernel - Added stone, sediment, snow, concrete and cement particle types - Added settings to scale the simulation for different hardware - Fixed movement of the nozzle in level to to be fps independent - Destroy objects that fall outside simulation bounds - Fix the ui issue to automatically scroll on different resolutions - Added steam explosion effect for level 2 overheated water gun - Added highlight when hitting target level 2 - Worked on poster for demo day - Added shortcut to rotate obstacles - Scaled the brush strength indicator in different resolutions - Added radioactive particles like plutonium and neutrons - Fixed issue where particles would fly in wrong direction - Made drawing particles more evenly distributed - Worked on level 3 where you mix lava to reach specific temperature - Completed level 5 where you pilot a spaceship to shoot lasers at a planet	**Game Features and Enhancements (Gravity mode and powder sections)** **Level 3** **Level 5**
Student B: (JM)		
Student C: (CTL)	- Level 4 - Puzzle Level - WIP: Level 5 Casting Level - Unified Shader Performance Benchmarks - Oral Presentation Slides	**Unified Physics Shader** **Level 4** **Casting Level**
Student D: (DC)	- Added Snapper script for snapping game objects together - Draggable script compatibility updates, max dragging speed limits - Added Cola, Whiskey fluid types - Added CupFactory script for cup cloning - Drink level development - Added FluidType detection to Fluid Detectors - Created new Fluid Detection parameters with public accessibility for game managers - Worked on poster for demo day - Oral presentation slides	**New scripts and script enhancements** **Drink mixing level** **Scripts refactoring**

Table 2: This is a breakdown of the tasks each member worked on **for this MCR period.**

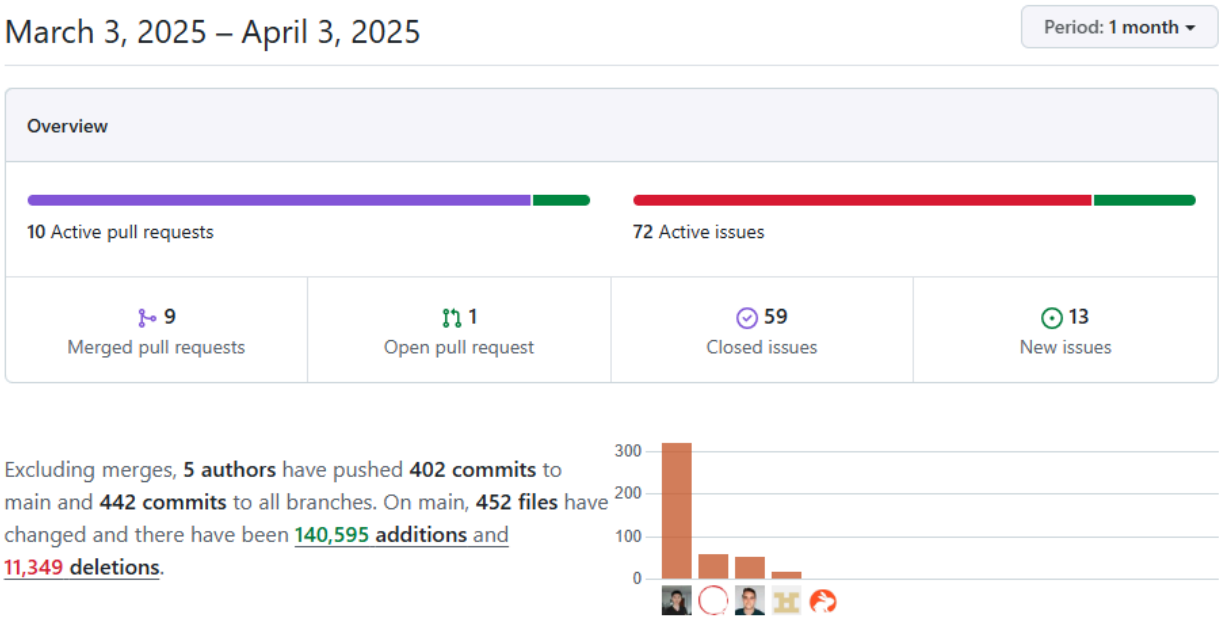
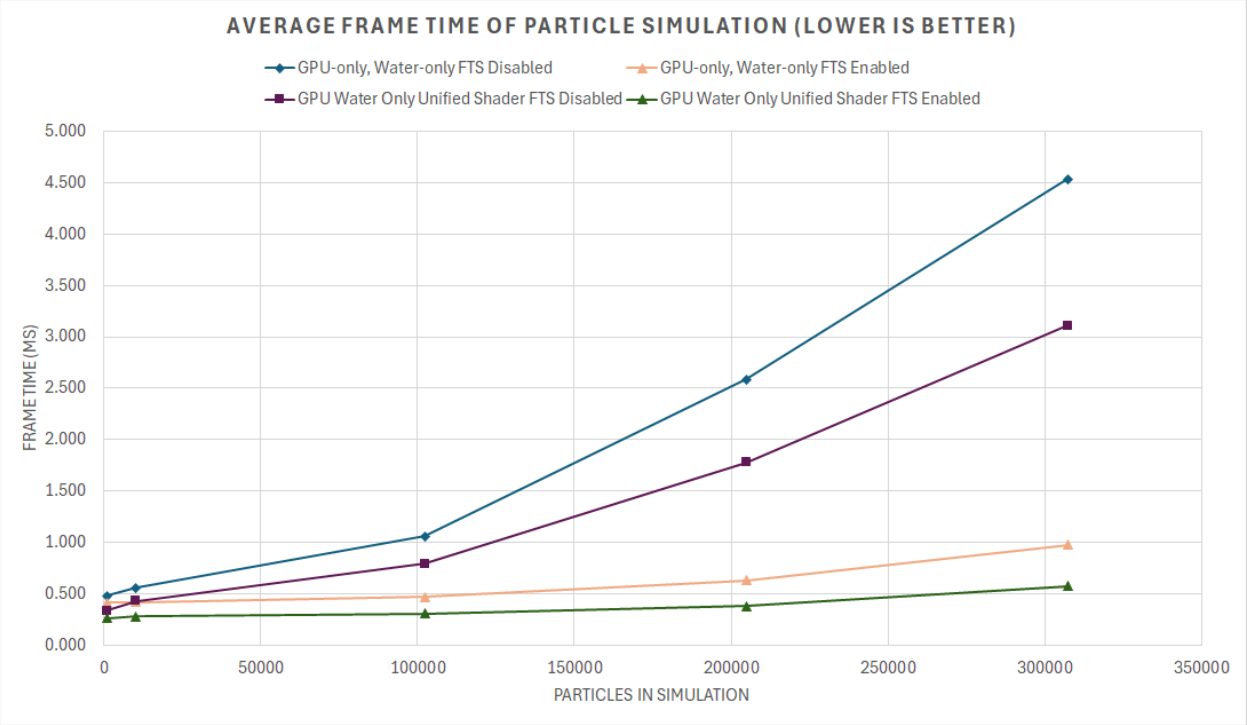


Figure X: This is a breakdown of closed issues and pull requests over the past month.

Game Features and Enhancements:

Unified Physics Shader

As we have increased the complexity of the fluid simulation physics such as adding friction and temperature, we have found that dispatching multiple separate shader files onto the GPU can have a significant impact on performance in high intensity scenes. In an effort to increase performance, the kernels for pressure, viscosity, friction, and temperature were combined into a single kernel. This has the effect of reducing the number of kernels dispatched onto the GPU, and makes management of the kernels simpler. It also reduces the total amount of memory reads needed for the broad-phase nearest neighbour checking algorithm. To determine if any performance uplift was generated, the unified kernel change was put through the standard test plan to measure performance.



Graph ##. Performance comparison between uniform and non-uniform compute shader implementations.

The unified shader showed improved performance when the fixed time step setting was enabled, and also showed a performance increase for large particle counts when the fixed time step setting was disabled. From the performance benchmarking results we have concluded that the unified shader is to be used on all levels in The Fluid Toy.

New Game Scripts and Script Enhancements

As part of a series of prerequisite features for one of the new levels in development, several scripts were added or enhanced to add some extra functionality. A universal game object “snapper” script for snapping one game object to another was added. This script allows level makers to designate one object as a target for another object to snap to, and when both

objects are brought together to the proximate snapping location, both objects will “snap” together, and the former will become a child object of the latter.

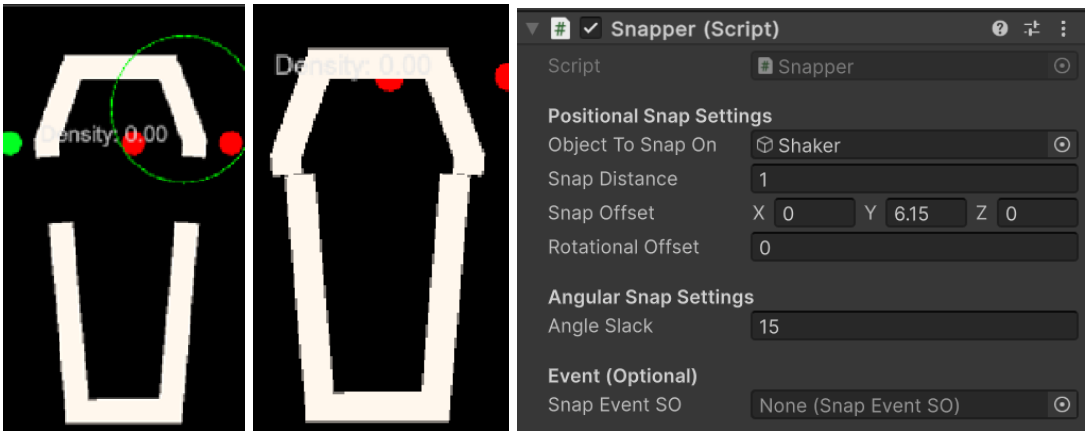


Figure X. (Left to right) Game object before/after snapping, Unity editor menu for Snapper

Another set of scripts were written to generate clones of game objects. The idea being that existing objects could be cloned into new ones using a factory script with a level-based trigger. The script then automatically injects collider and sensor data into the fluid simulation and any sensor managers, which ensures that particles are able to interact properly with the new cloned object. The trigger in this case is a button, and as seen in **Figure Y**, the factory is customizable and can generate a fixed number of game objects.

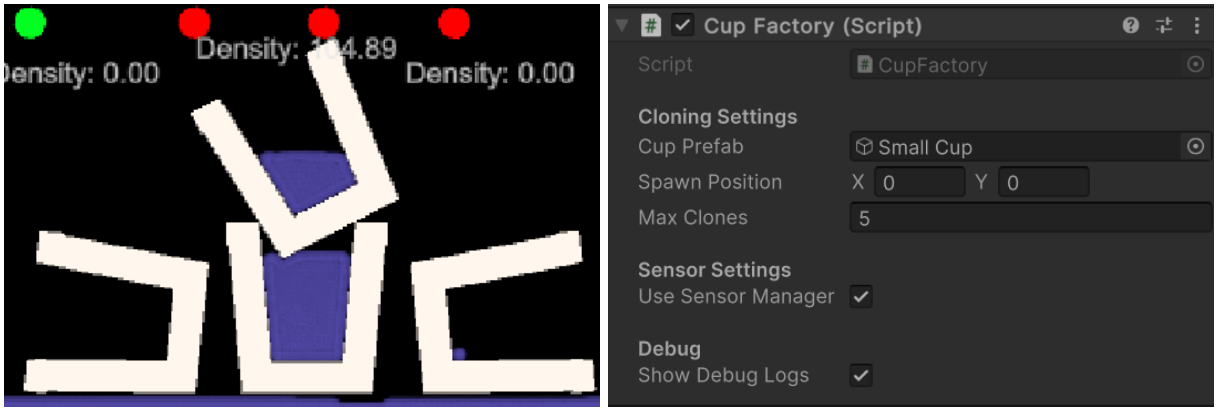


Figure Y. Stack of cloned cups, Unity editor menu for the cloning script

The existing Fluid Detector script was modified to enable typed fluid detection. The previous iteration of this script only parsed the fluid density of the region, whereas the enhancements added an “Type discrimination” flag, which allows the sensor to detect the type of particles within its range and expands the range of customization for fluid detection settings (Figure Z).

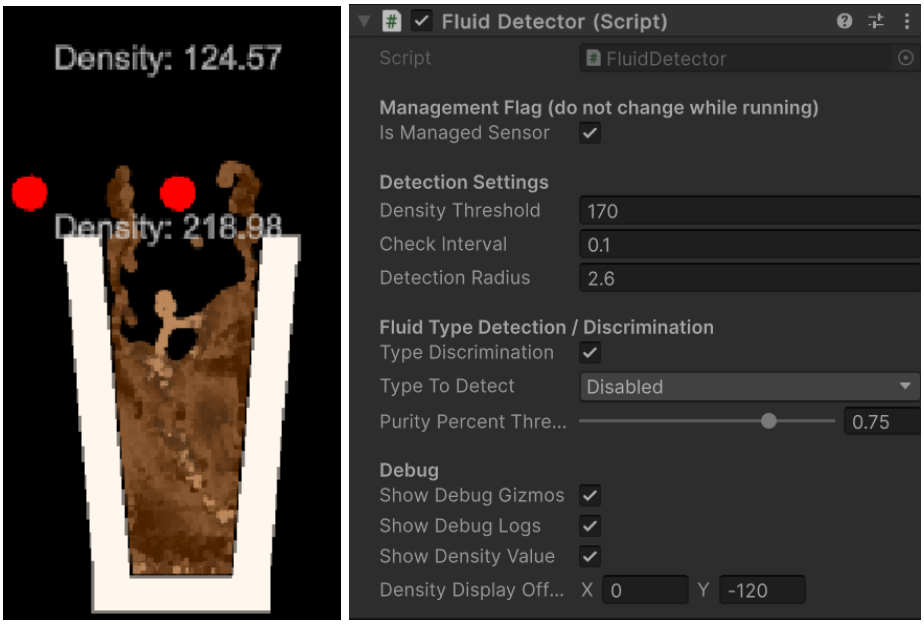


Figure Z. Two stacked Fluid Detectors in a cup on the left, Unity editor menu for Fluid Detectors featuring new options on the right.

We've added a gravity mode, which allows the game developer to set a direction for the gravity vector, they can make all particles experience reverse gravity, zero gravity and even make them attracted to the center of the screen which causes them to form planet like balls in the center of the screen and with enough velocity they are also able to form satellites which orbit that center of gravity.

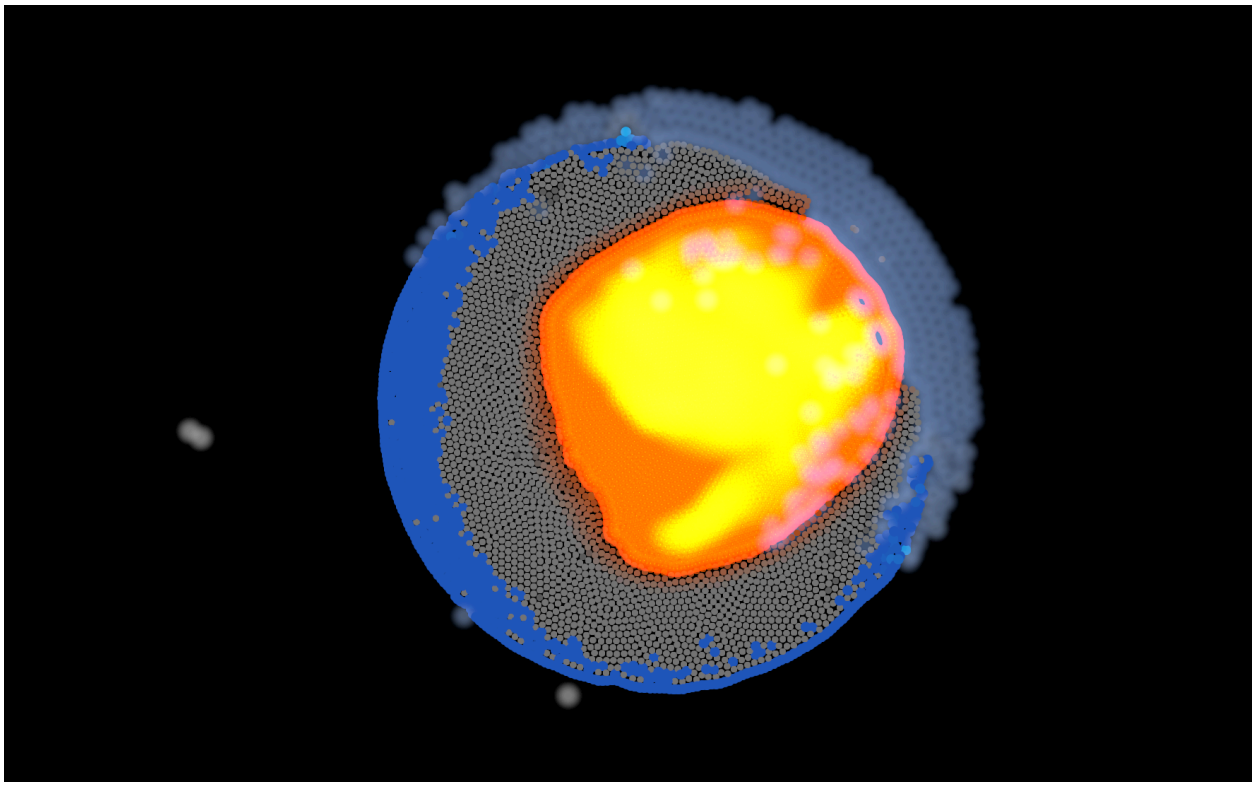


Figure Z. Image of planet with satellite, created using radial gravity

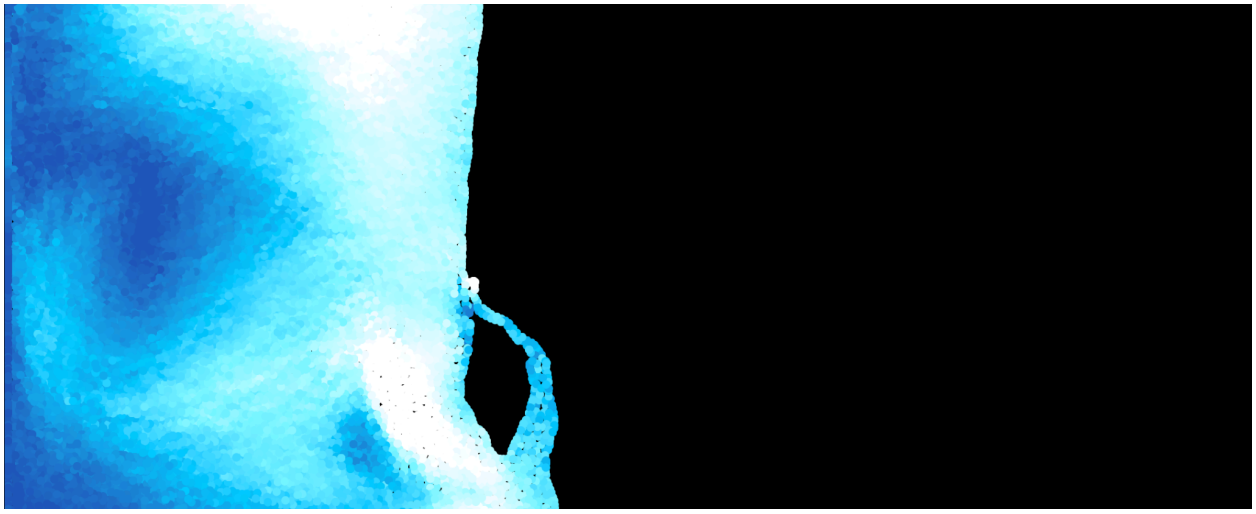


Figure Z. Image of water when the gravity direction is set to left

Powders were also added. Powders do not behave like other fluids, they have inter-particle friction which allows them to stack up and form piles. An example of this is snow, which can be stacked to form glaciers and another example of stone which behaves like many small gravel particles. This type of particle allows us to model water erosion and other geological phenomena.

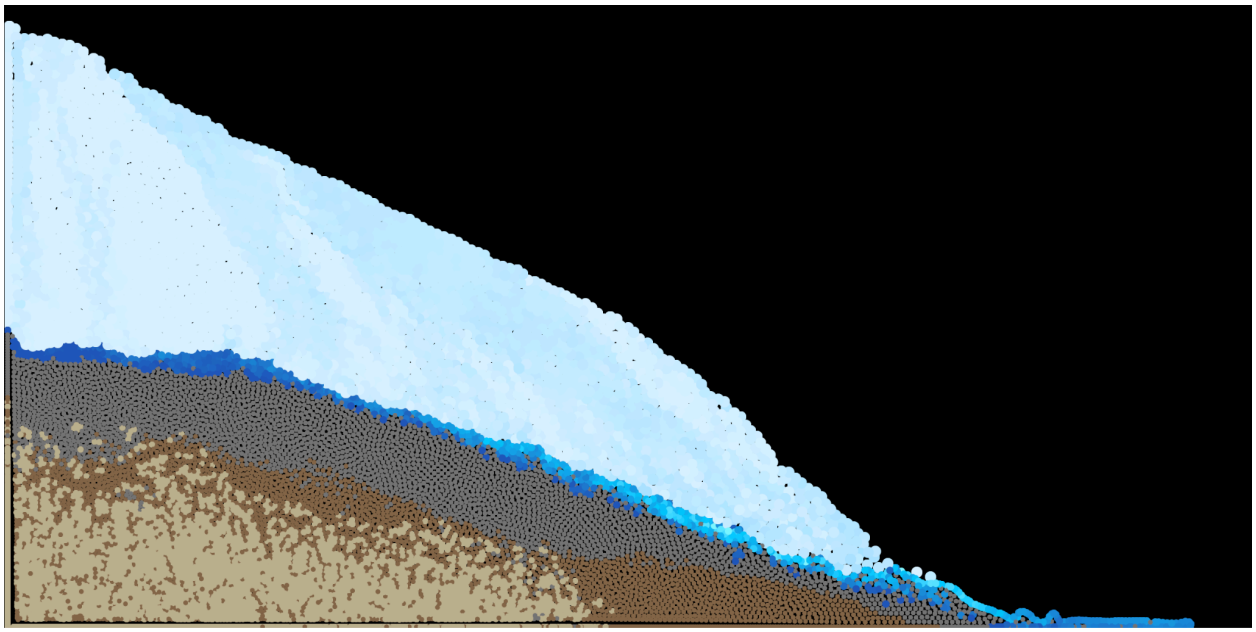


Figure Z. Image of snow pile ontop of stone pile

Solids have also been added. Solids like concrete, do not move like gasses, liquids, or powders. They stay in place. This allows the user to ‘draw’ shapes onto the screen which stay in place and allows developers more freedom to create destructible environments.

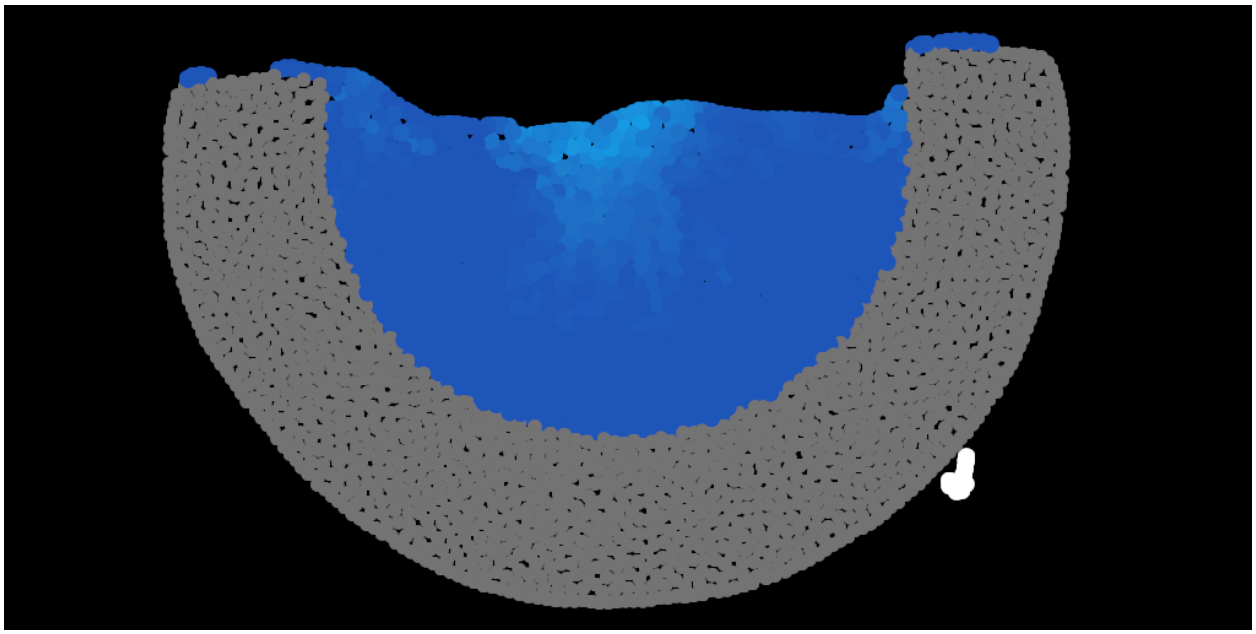


Figure Z. Image of concrete bowl filled with water

Game Content:

Level 3 Development

This is a very simple level where the goal is to mix different vats filled with lava until the desired temperature is reached. There are 4 different vats, but only 3 are filled with lava. The ones which are filled have 1200C, 1000C and 700C lava. The goal is to mix a combination of these to fill the remaining vat with 900C lava. This is a simple puzzle level that showcases our temperature simulation capabilities. When mixing an equal amount of particles of 1200C and 1000C for example the temperature will eventually average to 1100C.

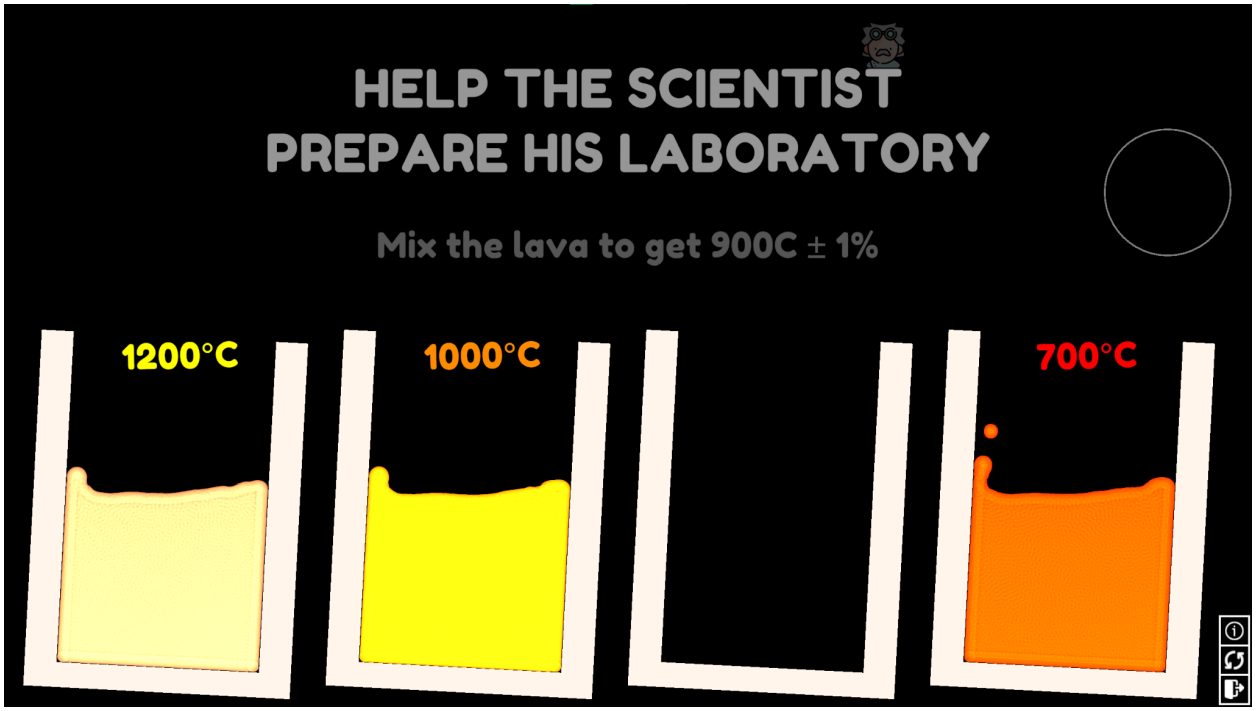


Figure ##. Level 3 - Lava Temperature Puzzle

Level 4 Development

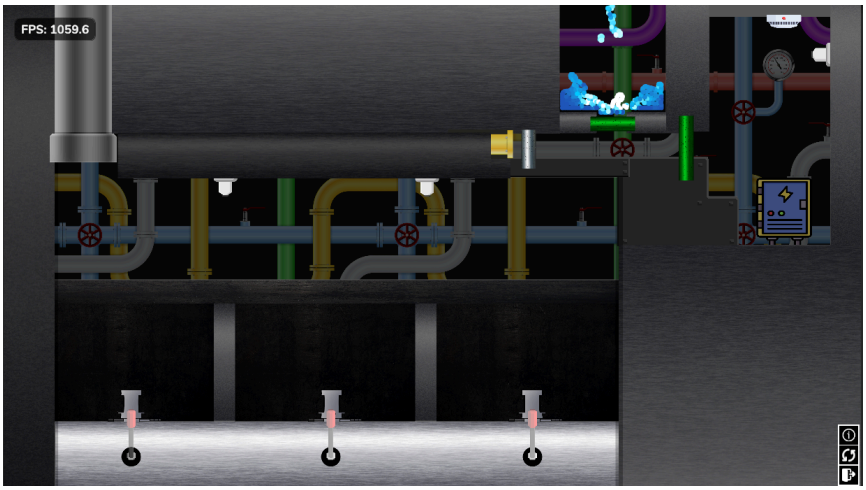


Figure ##. Level 4 - Fluid Puzzle

Level 4 is a puzzle solving level designed to showcase state changes in fluids. The player needs to find the button that lets them win the game. There are two interactable “gates” that the player can interact with, colored green. To complete the level the user must first pour water onto the electrical panel (pictured top right), which will cause a fire.

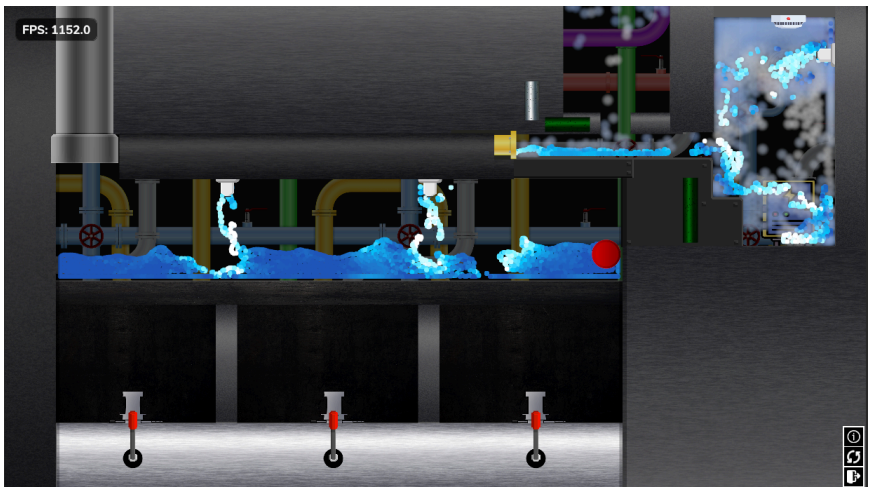


Figure ##. Level 4 - Water on the electrical panel causes a fire.

The fire from the electrical panel sets off the smoke detector (pictured in top right) and the sprinkler system activates, putting out the fire but also flooding the boiler chamber with water (pictured in the middle). The victory button can now be seen by the player as a red button in the bottom right corner of the boiler chamber. However, clicking the button will not complete the level until the water in the boiler chamber is removed.

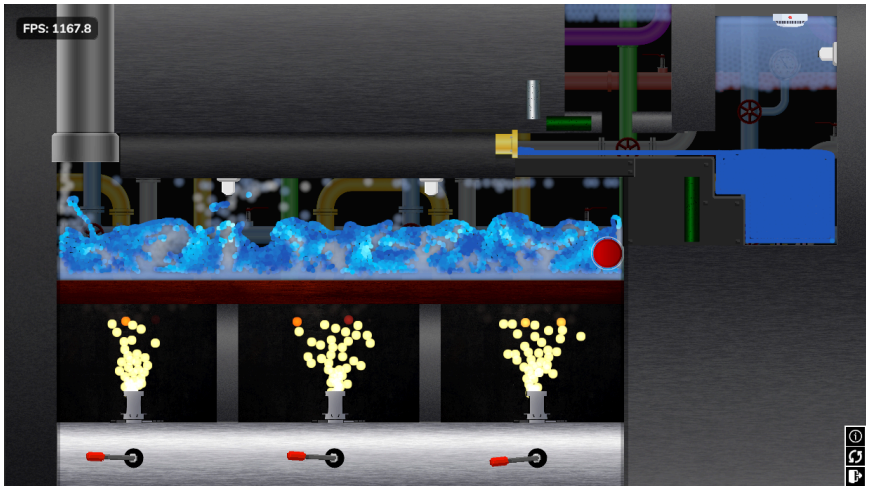


Figure ##. Level 4 - Burners causing water in the boiler chamber to turn to steam.

To boil away the water the player must precisely adjust the fuel valves (pictured at the bottom below the burners) until the bottom of the boiler chamber starts to heat up and the water begins to boil away. As the water boils away the victory button begins to turn green, indicating to the player that their actions are correct. Once the water and steam is sufficiently removed from the chamber, the button unlocks and the player pressed the button to win the level.

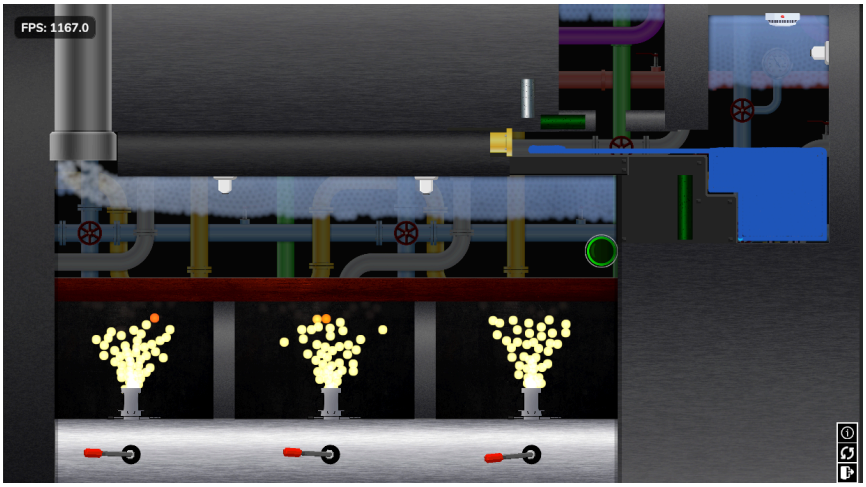


Figure ##. Level 4 - Water boiled to steam and vented out of the boiler chamber

This level was playtested amongst various friends to obtain player feedback and the overall feedback was positive, with most players interested in playing future levels for The Fluid Toy.

Level 5 Development

In this level the goal is to pilot a spaceship to destroy a planet with laser beams. A Panini projection is used to distort the perspective of the camera in order to create a pseudo 3D effect within our 2D fluid simulation engine. The laser beams are created using an application of many of the systems we have created up to this point, the source object spawns the particles and controls their initial velocity, the visual shader is stylized to make it look more like a laser beam and the physical properties of the particles which make up the laser beam are set up to melt and push away other particles that they come in contact with.

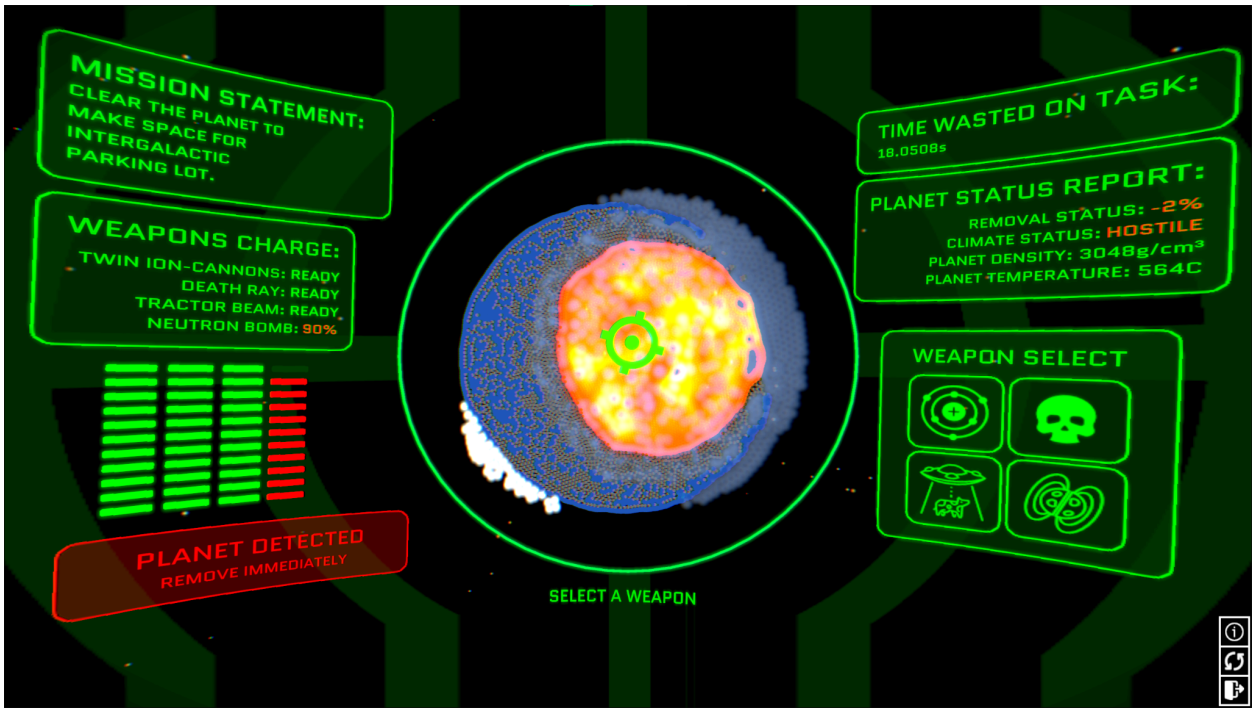


Figure ##. Level 5 - Image of the planet to be destroyed

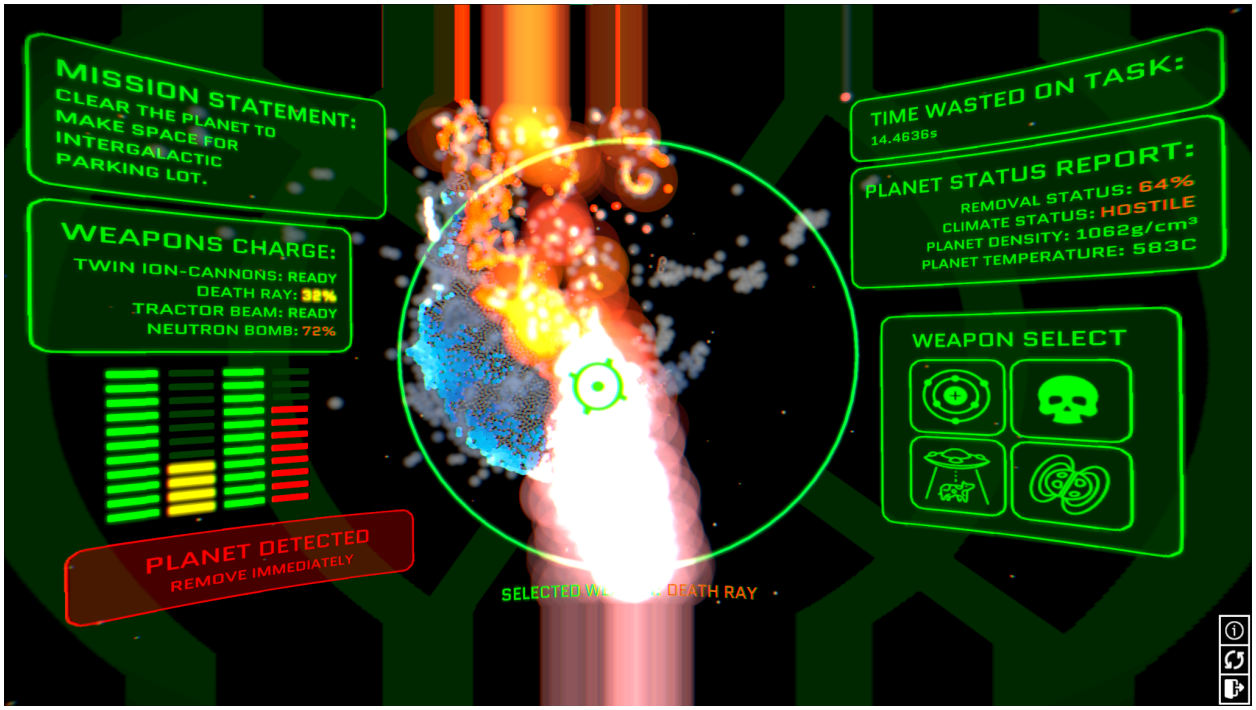


Figure ##. Level 5 - Image of the planet being destroyed by laser beam

The planet is formed using stone, water, snow and lava particles. The reason it stays as a ball in the center of the screen is because of the new gravity modes we added to the simulation engine. In one of the gravity modes called ‘Radial Gravity’ all the particles become attracted to the center of the screen, allowing them to form planets. This also enables them to orbit around the center of the screen as satellites.

Casting Level Development (Potential Level 6)

The casting level is designed to showcase The Fluid Toy's ability to simulate solids. The casting level has the player forge a sword from molten metal by pouring the metal into a cast and cooling the metal until it solidifies in the shape of the cast. The player is graded on their sword quality using density and thermal measurements to determine the quality of the casting.

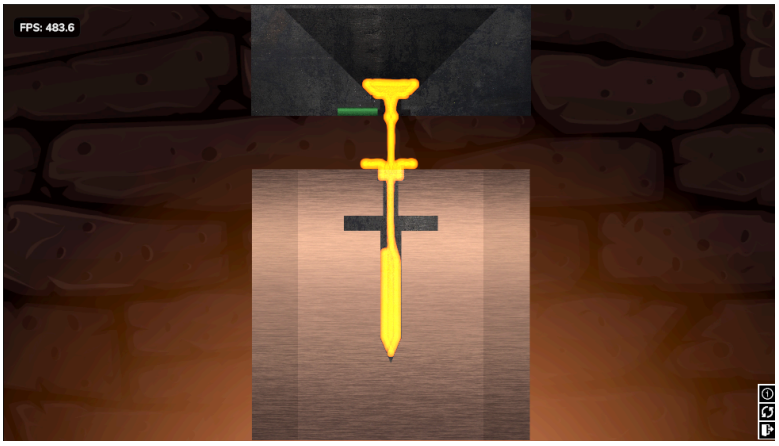


Figure ##. Casting Level - The cast is moved into position and the molten metal is poured into it.

First the metal is poured into the cast by the player. The player must take care not to spill any metal as that would result in a poor grade for the finished sword. Once all the metal is in the cast, the player needs to wait for the sword to cool. The player should avoid bumping or moving the cast as the sword is cooling as this can result in a poor quality result.

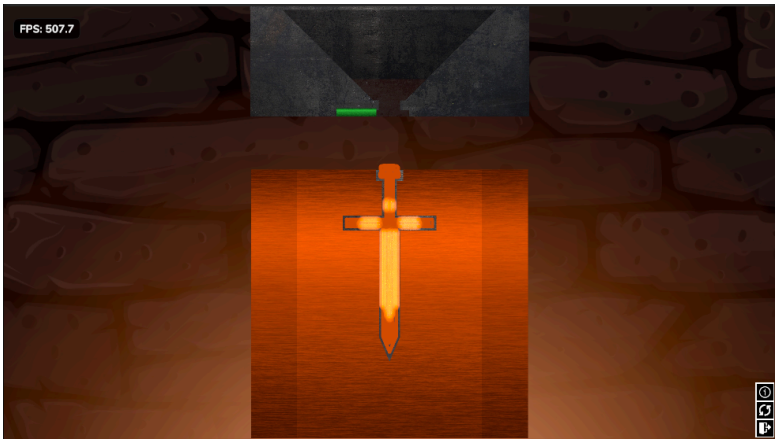


Figure ##. Casting Level - The sword cools and solidifies within the cast.

Once the sword has solidified, a lever will appear that will open the cast and drop the sword into a water basing to cool it.

Figure ##. Casting Level - Sword cooling in water basin

Finally the sword is removed from the basin once cooled and the player wins the level. The player's star count depends on the grade of the casting.

Drink Level Development

The drink level (better name pending) is a relatively simple level based on the idea of fast-food soda dispensing machines. The goal of the level is to fill cups of the correct size with the correct drink, placing filled cups in a dedicated scoring area to fulfill orders that appear on-screen. The level is currently mid-way through development, with fluid-dispensing buttons and cup-generating buttons having been implemented, alongside a skeleton level manager. Tasks to complete include generating a formal “order list” of drinks to complete, texturing, as well as implementing the formal scoring system. A screenshot of the level at its current stage in development may be seen below in **Figure A.**

Figure A. Early development-stage drink level.

Problems Encountered:

Scripts Refactoring

A number of existing scripts in our game will often conflict with new developments, or generally have strange artifacts within them due to rapid development or lack of foresight during their initial implementation. Examples include our fluid *SourceObject* script, which utilizes a simple integer for typing the fluid that is generated in lieu of the formal enumerator value, which actually describes which fluid is being mapped to (**Figure B**). This is relatively natural for game development, as foresight will usually be poor when the initial direction of the game or of specific technical elements are unclear and still amorphous, but also generates new work in the form of refactoring or overheads (i.e. do we refactor the integer to the enum, and fix it across all levels, potentially breaking many things, or do we take the overhead of having to manually map the fluid we wish to add to an integer?).

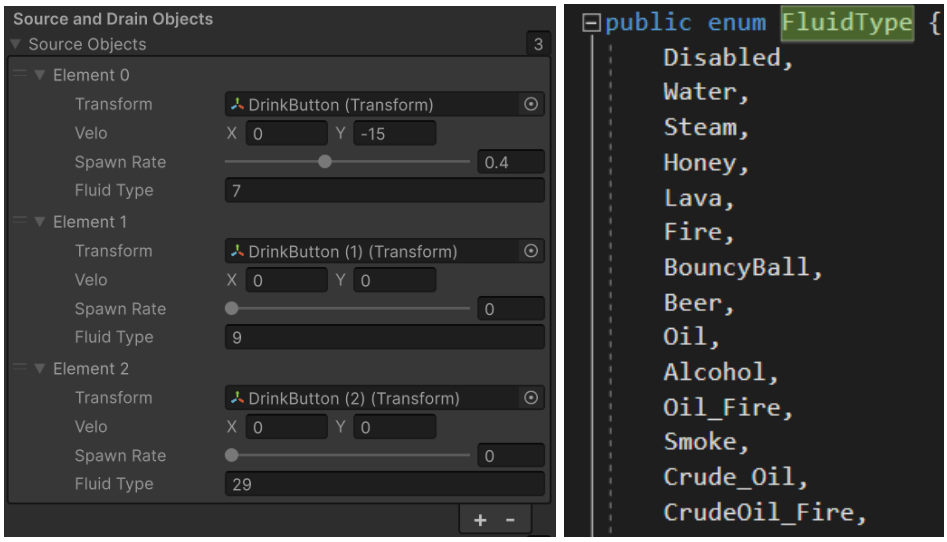


Figure B. SourceObject editor window and FluidType enum. The “Fluid Type” integer field maps to the FluidType enum on the right, requiring manual counting to be set in the Unity editor.

Other examples include the *Draggable* script, which enables solid physics objects to be dragged. In this case, the conflict of functionality was not clear at all until the new feature, *Snapping*, was to be added. Both scripts try to manipulate the position of game objects at the same time, leading to a natural conflict in functionality between the two. Luckily, fixing the problem here simply required lifting of the *isDragging* class member to the public space in order to resolve the technical block, but future fixes might not always be so simple.

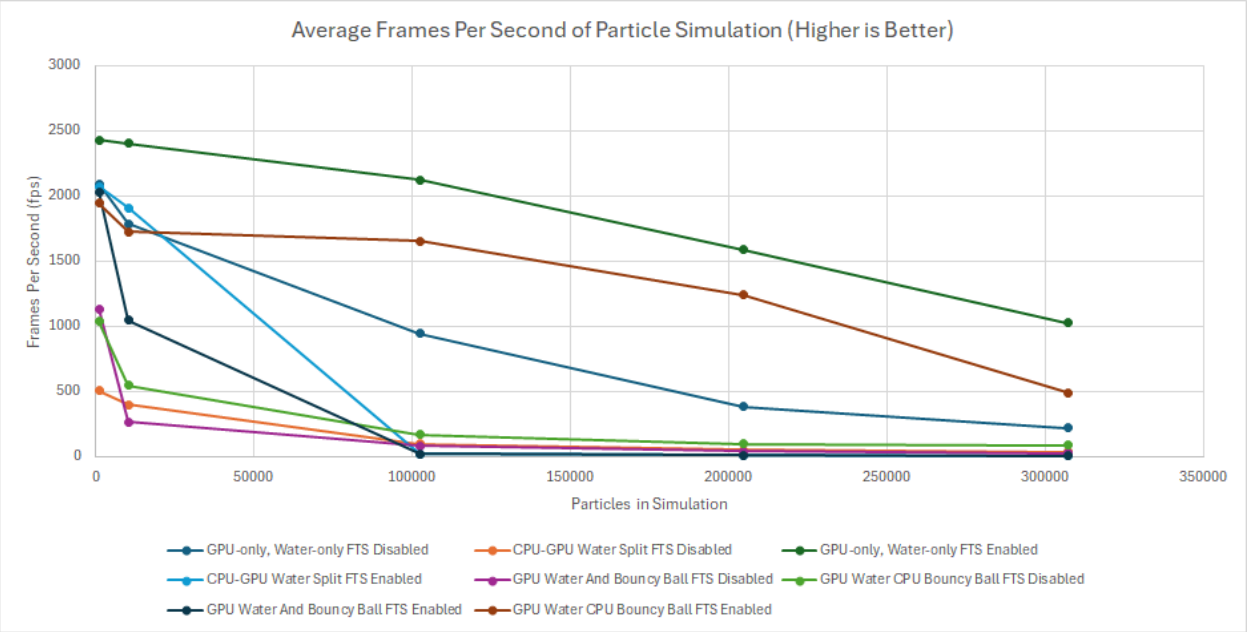
When testing the web application on a friend’s iOS device, performance issues were encountered. It is difficult to debug these issues because none of us have personal iOS phones for testing. We have a single MacOS device which we have been using to debug the issue for that platform but lack of hardware continues to pose a challenge.

Conclusion:

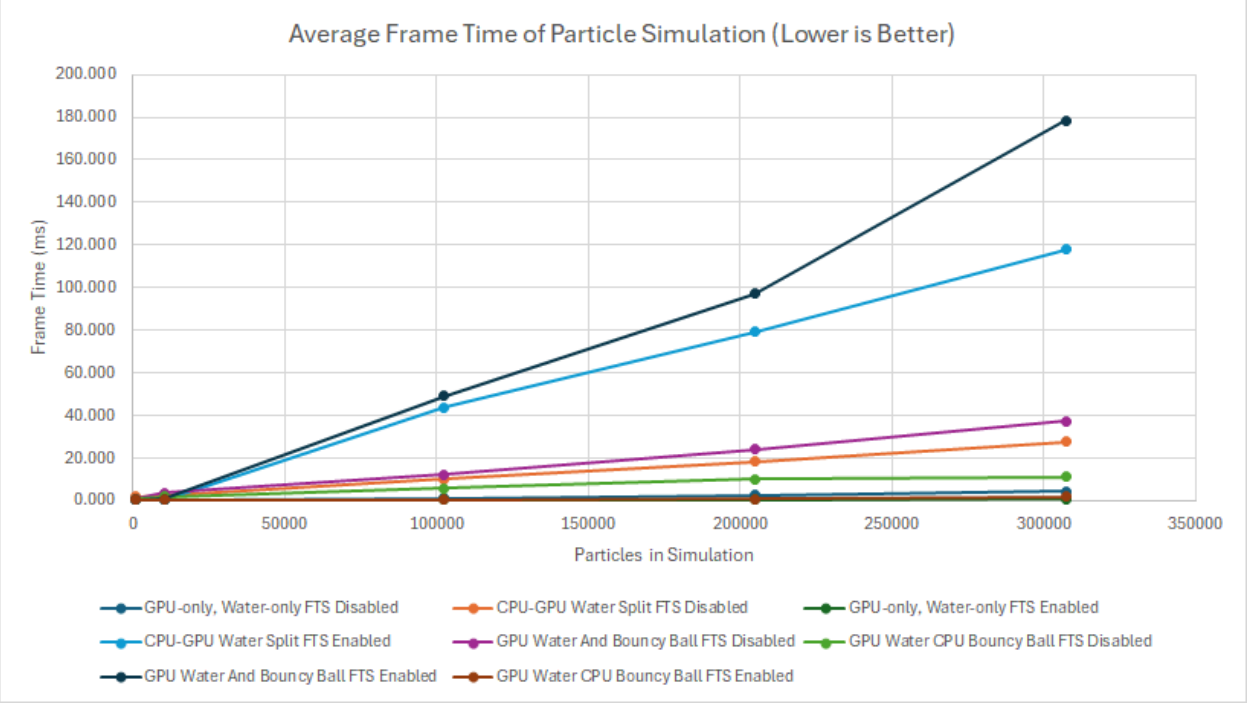
This milestone marks significant progress towards the completion of the fluid simulation capstone project. The integration of a unified physics shader, coupled with new game mechanics centered around gravity modes and powders, has substantially improved the simulation's realism and performance. The development of new levels showcases the breadth of possibilities enabled by the simulation engine. Ongoing work involves level completion and addressing technical challenges, such as refactoring and hardware incompatibilities. The project remains on track to deliver a compelling and feature-rich demonstration of fluid simulation capabilities.

Appendix:

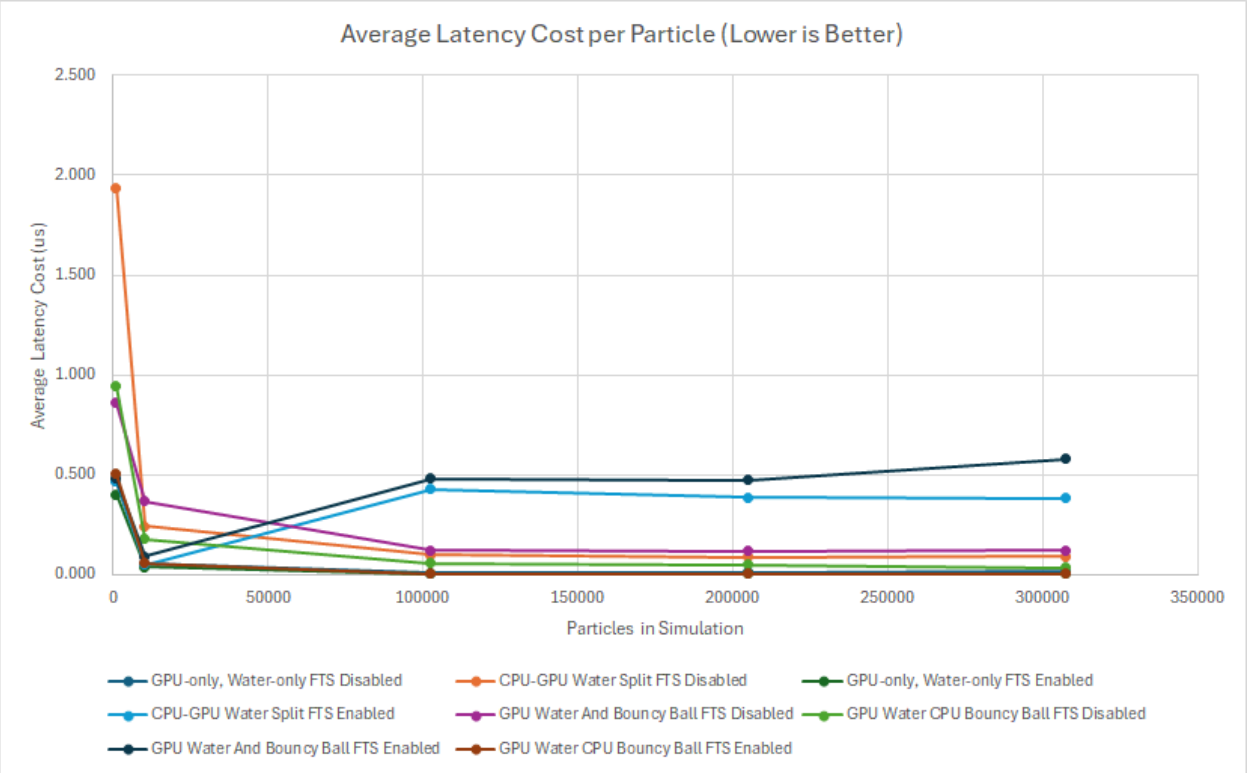
Graph A-1: Performance Benchmarking Results - Average Frames Per Second (fps)



Graph A-2: Performance Benchmarking Results - Average Frame Time (ms)



Graph A-3: Performance Benchmarking Results - Average Latency Cost Per Particle (us)



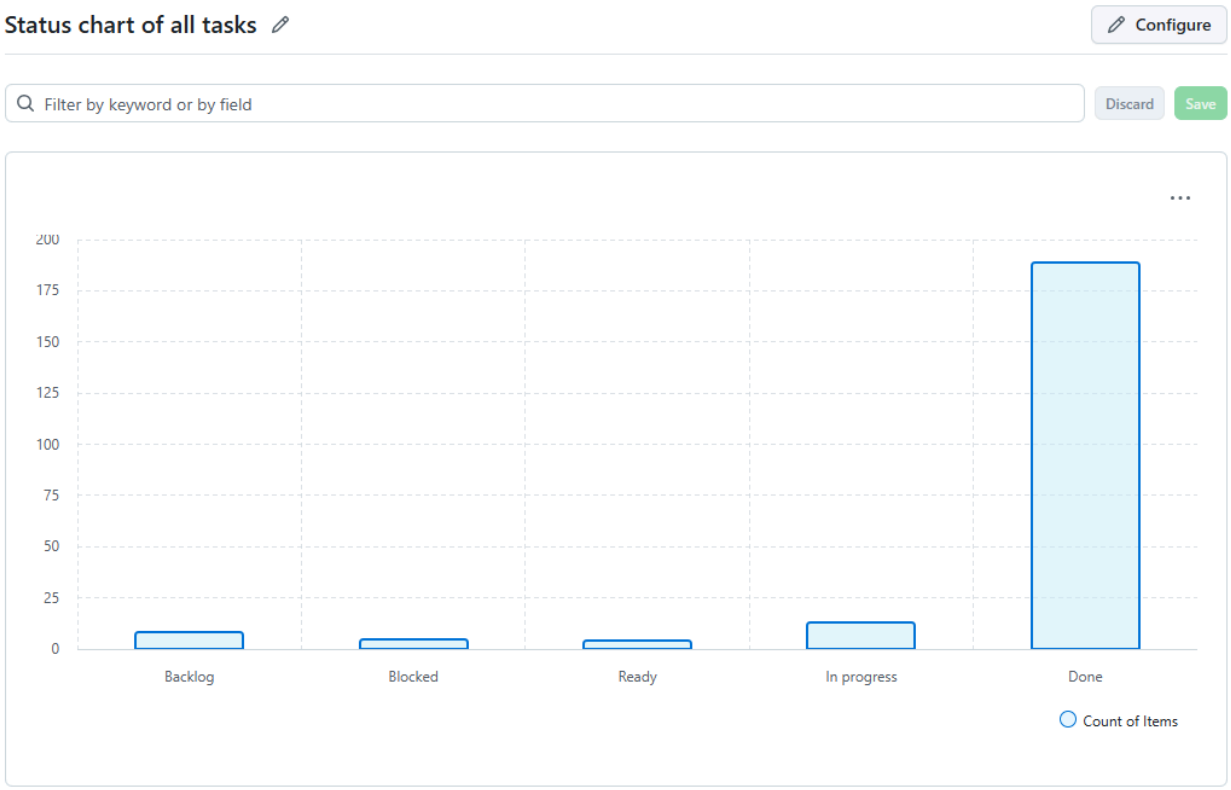


Figure X: The current status of all tasks on our github repository.
(More Tasks will be created as is necessary)

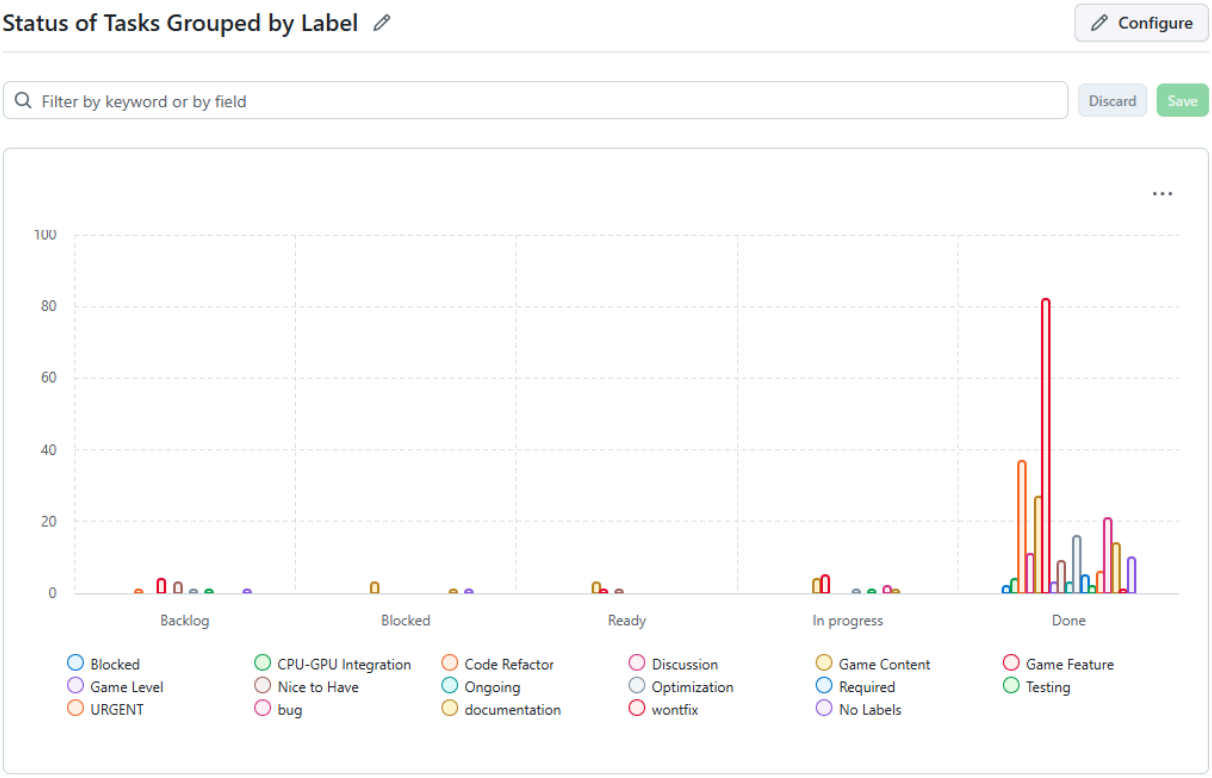


Figure X: This is the breakdown of the current status of all tasks based on their type. Each bar from left to right corresponds to the legend labels from left to right.

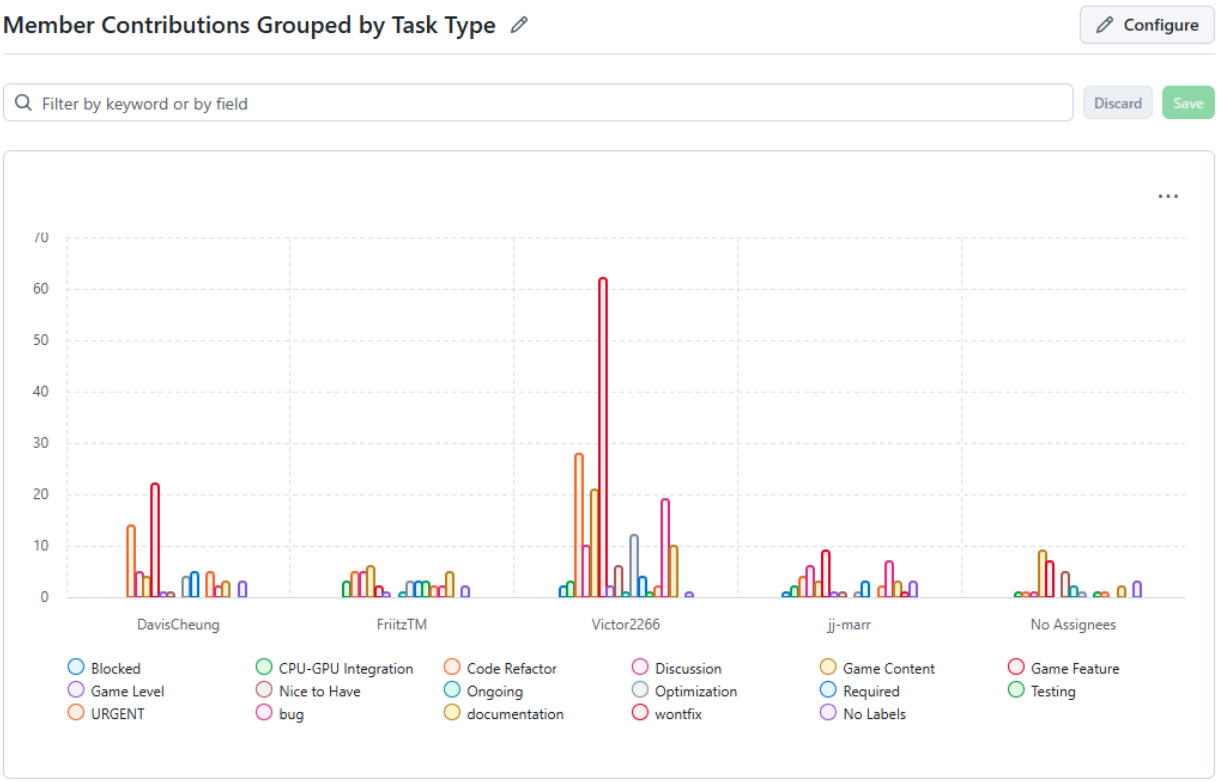


Figure X: This is a breakdown of the types of tasks each member has worked on grouped by task type.

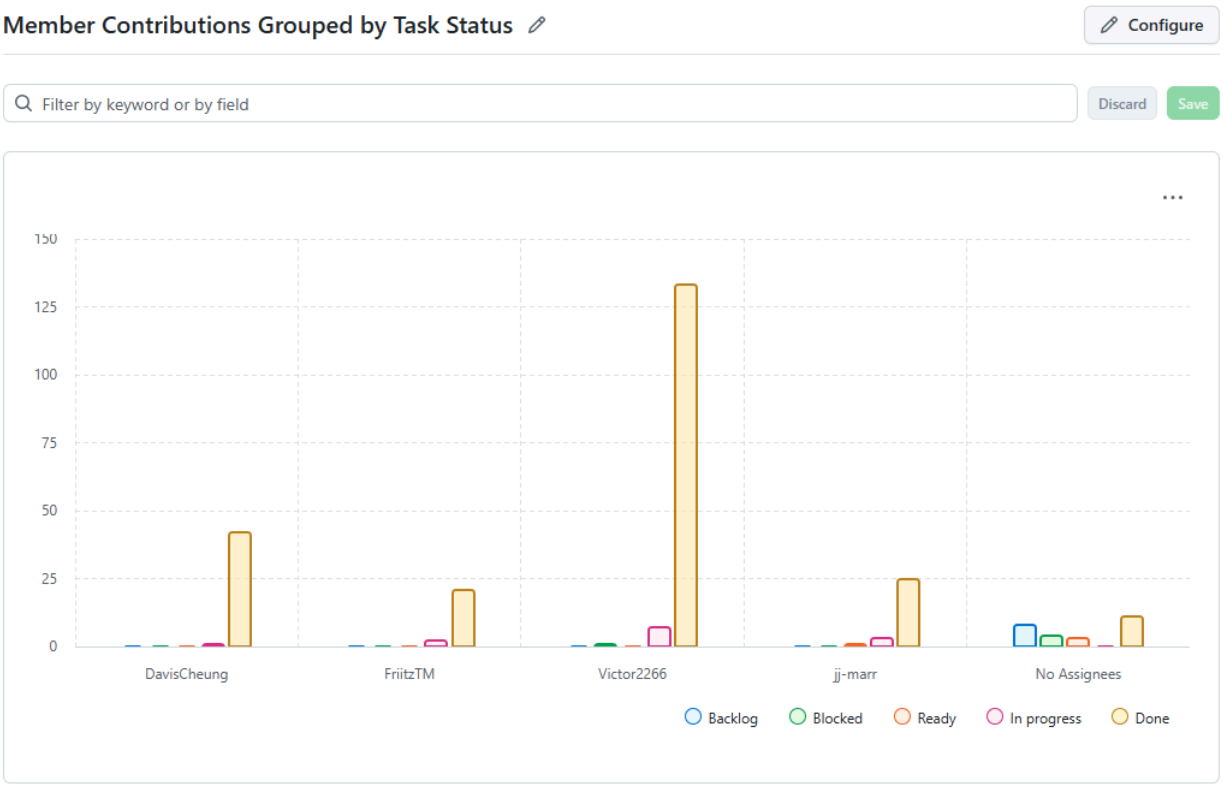


Figure X: This is a breakdown of the status of tasks assigned to each member.

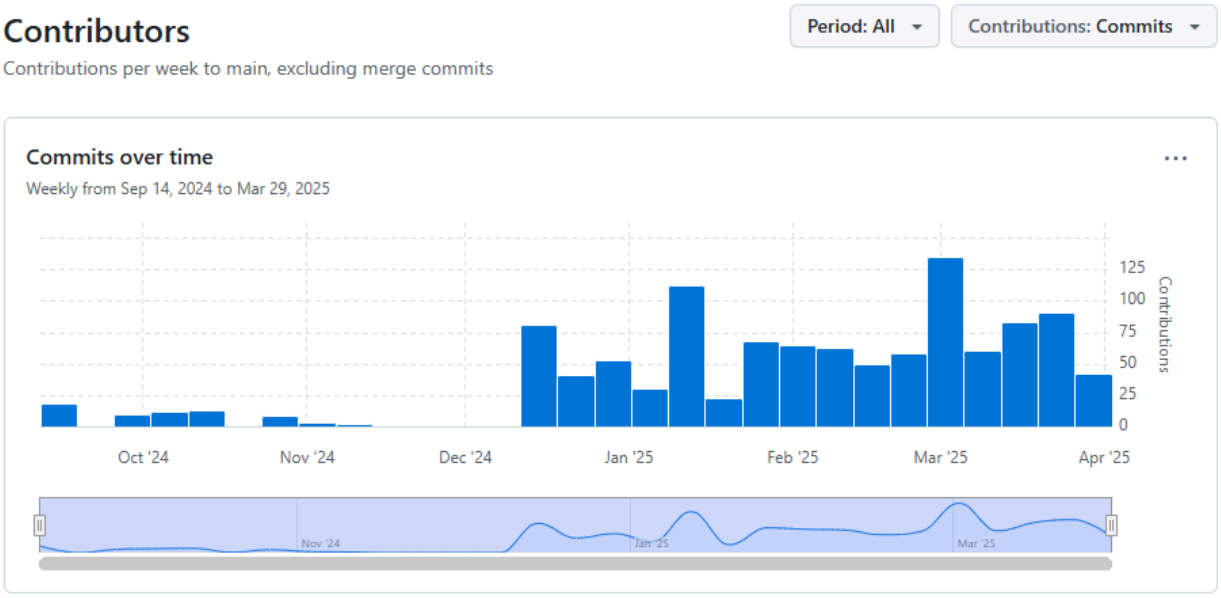


Figure X: This is the commit history of the project.

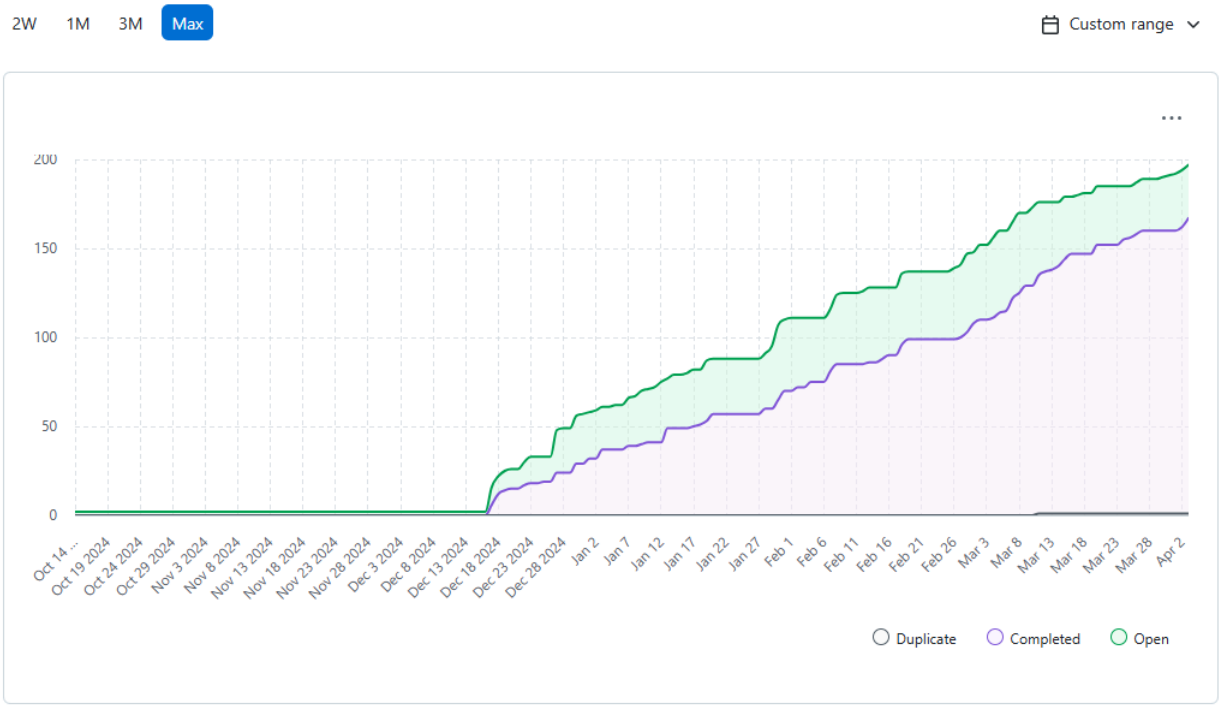


Figure X: The Burn up chart shows the progress of the project items over time, showing how much work has been completed and how much is left to do.This showcases how we continually open and close new issues with an average of 40 open issues at any given time.